

LC 1314: Matrix Block Sum

Problem Notes & Optimized Solution Implementation

Google

Amazon

Meta

Problem Understanding

For every cell (i, j) , we need the sum of all cells within a distance of k .

1. Brute Force Approach

Iterate through every cell, and for each cell, iterate through all rows and columns inside the k -distance block.

```
for every cell {
    for all rows inside block {
        for all cols inside block {
            sum += matrix[row][col]
        }
    }
}
```

TC = $O(n \times m \times k^2)$

2. Optimized Method: Prefix Sum

A better method is to build a prefix sum array and then compute the required sub-matrix sum in constant time.

Boundary Management

To avoid out-of-bounds errors when determining the block around (i, j) :

```
r1 = max(0, i - k); // Maximize to avoid out of bounds (negative indices)
```

```
c1 = max(0, j - k);
```

```
r2 = min(n - 1, i + k); // Minimize as it can go out of bounds (exceed dimensions)
```

```
c2 = min(m - 1, j + k);
```

TC = $O(n \times m)$

SC = $O(n \times m)$

C++ Implementation

```

class Solution {
public:
    vector<vector<int>> matrixBlockSum(vector<vector<int>>& mat, int k)
    {
        int n = mat.size();
        int m = mat[0].size();
        vector<vector<int>> pref(n, vector<int>(m));

        // 1. Build the Prefix Sum Array
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                pref[i][j] = mat[i][j];
                if(i > 0) {
                    pref[i][j] += pref[i-1][j];
                }
                if(j > 0) {
                    pref[i][j] += pref[i][j-1];
                }
                if(i > 0 && j > 0) {
                    pref[i][j] -= pref[i-1][j-1];
                }
            }
        }

        vector<vector<int>> ans(n, vector<int>(m));

        // 2. Calculate the block sum for each cell
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                int r1 = max(0, i - k);
                int r2 = min(n - 1, i + k);
                int c1 = max(0, j - k);
                int c2 = min(m - 1, j + k);

                int sum = pref[r2][c2];
                if(r1 > 0) {
                    sum -= pref[r1-1][c2];
                }
                if(c1 > 0) {
                    sum -= pref[r2][c1-1];
                }
                if(r1 > 0 && c1 > 0) {
                    sum += pref[r1-1][c1-1];
                }

                ans[i][j] = sum;
            }
        }

        return ans;
    }
};

```